DevOps Interview Questions & Answers

A reference guide covering Docker/Shell scripting, Jenkins, Argo CD, Git, Kubernetes, and Terraform.

Contents

- Docker / Shell / SSH Scripting
- Jenkins & Argo CD
- Git
- Kubernetes
- Terraform

Docker / Shell / SSH Scripting

Q: Write a script to check whether a Docker container named mydb is running on hosts host1 to host10 using SSH.

A: Loop over the host list, SSH into each one, and query Docker's running-container list filtered by name:

```
#!/bin/bash
for i in $(seq 1 10); do
  host="host$i"
  status=$(ssh -o ConnectTimeout=5 "$host" "docker ps --filter 'name=^/mydb$' --format '{{.Names}}'" 2>/dev/null)
  if [ "$status" == "mydb" ]; then
    echo "$host: mydb is RUNNING"
  else
    echo "$host: mydb is NOT running"
  fi
done
```

Q: Can you explain what you have written in the Docker/SSH script?

A: The script loops through `host1` to `host10` by generating the range with `seq`. For each host it runs `docker ps` remotely over SSH, filtering by the exact container name `mydb` using a regex anchor (`^/mydb$`) so it doesn't match partial names like `mydb-backup`. The `--format '{{.Names}}'` flag returns just the container name instead of the full table, which makes it easy to compare against the expected value in the `if` statement. `ConnectTimeout=5` prevents the script from hanging on unreachable hosts, and stderr is suppressed so SSH connection failures don't clutter the output — a host that fails to connect simply falls into the "not running" branch.

Q: Filter the numbers 1, 2, 5, 7, 10, 8, 19, find the odd numbers and return their squares.

A: Odd numbers: 1, 5, 7, 19. Their squares: `1 → 1`, `5 → 25`, `7 → 49`, `19 → 361`.

```
nums = [1, 2, 5, 7, 10, 8, 19]
result = [n**2 for n in nums if n % 2 != 0]
print(result)  # [1, 25, 49, 361]
```

Q: Are you more comfortable with Python or Shell scripting?

A: This is a personal/experience question best answered from your own background. A strong answer typically states a primary comfort level while showing versatility — e.g., "I'm more comfortable with Shell for day-to-day operational tasks like log parsing, service checks, and CI glue scripts, but I use Python when the logic gets more complex — data structures, error handling, or API calls — since it's easier to maintain and test."

Q: Write the odd-number/square logic in Shell without mixing it with Python.

```
#!/bin/bash
nums=(1 2 5 7 10 8 19)
result=()
for n in "${nums[@]}"; do
  if (( n % 2 != 0 )); then
    result+=($(( n * n )))
  fi
done
echo "${result[@]}"   # 1 25 49 361
```

Q: Print the 10th and 15th characters of a string using Shell/Linux commands.

A: Using Bash parameter expansion (0-indexed, so subtract 1) or `cut`:

```
str="ThisIsASampleStringForTesting"

# Bash substring expansion
echo "${str:9:1}"    # 10th character
echo "${str:14:1}"   # 15th character

# Equivalent using cut
echo "$str" | cut -c10
echo "$str" | cut -c15
```

Jenkins & Argo CD

Q: How are you using Jenkins and Argo CD together?

A: A common pattern is GitOps-based CI/CD split across two tools: Jenkins handles the CI side — checking out code, running tests, building the Docker image, pushing it to a registry, and then updating the image tag in a Kubernetes manifest or Helm values file stored in a Git repo. Argo CD then handles CD — it watches that Git repo and automatically (or on approval) syncs the new manifest to the cluster, so the actual deployment step is decoupled from Jenkins and driven purely by the desired state in Git.

Q: What if you want to roll back in Argo CD?

A: Since Argo CD tracks application state as a history of synced Git revisions, rolling back means either reverting the commit in the Git repo (the GitOps-correct way, since Argo CD self-heals to match Git) or using Argo CD's own history to sync back to a previous revision directly from the UI or CLI.

Q: If you want to perform the rollback through Argo CD, what command would you use?

```
# List revision history for an app
argocd app history <app-name>

# Roll back to a specific revision ID from that history
argocd app rollback <app-name> <revision-id>
```

Q: Have you heard about Jenkins Shared Libraries?

A: Yes — Shared Libraries are reusable Groovy code (custom steps, classes, and pipeline logic) stored in a separate Git repository and imported into Jenkinsfiles with `@Library('my-shared-lib') _` , rather than duplicating pipeline logic across every job.

Q: Why are Jenkins Shared Libraries used?

A: They centralize common pipeline logic — build steps, deployment routines, notification/Slack integration, versioning conventions — so that dozens of Jenkinsfiles across different services can call the same tested code instead of copy-pasting it. This makes pipelines DRY, easier to maintain (fix a bug once, it propagates everywhere), enforce organization-wide standards, and keeps individual Jenkinsfiles short and declarative.

Git

Q: What is git rebase?

A: `git rebase` replays the commits from one branch on top of another branch's tip, effectively rewriting commit history to create a linear sequence rather than a merge commit. For example, `git rebase main` while on a feature branch takes your feature commits and reapplies them as if they were made starting from the latest point on `main`.

Q: How is git rebase different from other Git approaches such as merge?

A: `git merge` combines two branch histories by creating a new merge commit that has two parents, preserving the exact history of both branches (including all the back-and-forth). `git rebase` instead rewrites history by moving your commits to start from a new base, producing a clean, linear history with no merge commits. The trade-off: rebase gives a cleaner log but rewrites commit SHAs, which is risky on shared/public branches (it breaks history for anyone else who already pulled those commits), whereas merge is non-destructive and safe to use on shared branches at the cost of a messier graph.

Kubernetes

Q: How do you deploy and manage your applications in Kubernetes? Are you using microservices or legacy applications?

A: This is a project-experience question. A solid structure for your answer: describe the application architecture (microservices vs. monolith), the deployment mechanism (Helm charts, Kustomize, or raw manifests, typically deployed via a GitOps tool like Argo CD or Flux), and how you manage config/secrets (ConfigMaps, Secrets, or a vault integration), plus how you handle scaling (HPA) and resilience (readiness/liveness probes, PodDisruptionBudgets).

Q: What is Kubernetes Ingress?

A: Ingress is a Kubernetes API object that manages external HTTP/HTTPS access to services inside the cluster, providing a single entry point that can do host- and path-based routing, TLS termination, and load balancing — instead of exposing every service individually via NodePort or a separate LoadBalancer.

Q: How does Ingress work?

A: An Ingress resource defines routing rules (e.g., `/api` → service A, `/app` → service B), but it does nothing on its own — it requires an Ingress Controller (NGINX, Traefik, ALB Ingress Controller, etc.) running in the cluster to actually watch Ingress objects and program a reverse proxy/load balancer accordingly. Traffic flow: client request → external load balancer/DNS → Ingress Controller pod → matched based on Ingress rules → forwarded to the correct Kubernetes Service → Service load-balances across matching pod endpoints via kube-proxy.

Q: If an application is hosted in Kubernetes, explain the complete traffic flow from the user clicking the application until it reaches the pod.

A: 1) User's browser resolves the domain via DNS to a public IP, usually a cloud load balancer. 2) The cloud Load Balancer (e.g., AWS ALB/NLB) forwards the request to a node in the cluster, often via the Ingress Controller's Service (typically type LoadBalancer or NodePort). 3) The Ingress Controller pod receives the request and matches it against Ingress rules (host/path). 4) It forwards the request to the matched Kubernetes Service (a virtual, stable IP/DNS name). 5) kube-proxy (via iptables/IPVS rules) load-balances the request across the healthy pod endpoints registered to that Service. 6) The request reaches a specific pod's container, is processed by the application, and the response travels back along the same path.

Q: Who manages your Kubernetes clusters — AWS or your own team?

A: This depends on your setup. If using a managed service like EKS, AWS manages the control plane (API server, etcd, scheduler) while your team manages worker nodes, workloads, and add-ons. If self-managed (kubeadm on EC2, for example), your own team is responsible for the entire cluster including the control plane. Answer based on your actual environment.

Q: If you want to send 60% traffic to one subset/version of the application, how would you do it? What deployment strategy are you using?

A: This is a Canary deployment. It's typically implemented not with vanilla Kubernetes Services (which only do equal-weight round robin) but with a service mesh or ingress-level traffic splitting tool — Argo Rollouts, Istio VirtualService weighted routing, or an NGINX Ingress Controller with canary annotations — where you explicitly define weighted routing, e.g., 60% to the stable version and 40% to the canary, and shift the percentage gradually as confidence grows.

Q: Where do you write/configure the deployment strategy?

A: Depending on the tooling: in an Argo Rollouts `Rollout` CRD manifest (specifying canary steps and traffic weights), in an Istio `VirtualService` / `DestinationRule` YAML (specifying subset weights), or in NGINX Ingress annotations (`nginx.ingress.kubernetes.io/canary-weight`). These manifests live in the same Git repo as the rest of your Kubernetes config and are applied/synced the same way (kubectl apply, Helm, or GitOps via Argo CD).

Q: What are probes in Kubernetes?

A: Probes are health checks the kubelet performs on containers to decide how to manage them. There are three types: **liveness probe** (is the container still healthy? if it fails, the container is restarted), **readiness probe** (is the container ready to accept traffic? if it fails, the pod is removed from the Service's endpoint list without being restarted), and **startup probe** (has the application finished starting up? other probes are disabled until this succeeds).

Q: What is a startup probe?

A: A startup probe checks whether a slow-starting container has finished initializing before liveness and readiness probes take over. It's useful for legacy or heavy applications that take a long time to boot — without it, a liveness probe with a short timeout might kill the container repeatedly during a legitimately slow startup. Once the startup probe succeeds, Kubernetes hands off health checking to the regular liveness/readiness probes.

Q: What are the main components of Kubernetes?

A: **Control plane:** kube-apiserver (front door for all API requests), etcd (key-value store holding cluster state), kube-scheduler (assigns pods to nodes), kube-controller-manager (runs controllers that reconcile desired vs. actual state), and cloud-controller-manager (integrates with the cloud provider). **Node components:** kubelet (agent that ensures containers are running as specified), kube-proxy (manages network rules for Service routing), and the container runtime (containerd, CRI-O, etc.).

Q: Suppose the entire Kubernetes cluster is down. How would you troubleshoot and recover it?

A: Systematically work from the bottom up: 1) Check the underlying infrastructure — are the nodes/VMs even running (cloud console, ping, SSH)? 2) Check control plane health — is the API server reachable (`kubectl cluster-info` , or direct `curl` to the API server if kubectl itself can't connect)? 3) Check etcd — is it healthy and reachable, since the API server depends on it entirely? 4) Check kubelet status on nodes (`systemctl status kubelet`) and container runtime status. 5) Review logs on control plane components (kube-apiserver, etcd, scheduler, controller-manager) for crash loops, certificate expiry, or disk pressure. 6) Once root cause is found (common causes: expired certs, etcd disk full, network/security group misconfiguration, a bad manifest applied to a critical component), fix it and restart components in dependency order: etcd → API server → scheduler/controller-manager → kubelet.

Q: What is your mindset/step-by-step approach when users cannot access an application because the entire cluster is down?

A: Stay calm and work top-down from user impact to root cause: first confirm the scope (is it one app, one namespace, or the whole cluster?), check monitoring/alerting dashboards for a starting point, verify basic connectivity (DNS, load balancer, Ingress), then drill into cluster health (API server, nodes, etcd). Communicate status to stakeholders early, avoid making unverified changes under pressure, and once mitigated, do a root-cause analysis and write a postmortem so the same failure doesn't recur.

Q: What are the minimum required Kubernetes components needed to bring the cluster back?

A: At minimum: a healthy **etcd** (cluster state must exist and be reachable), a running **kube-apiserver** (so kubectl and all other components can communicate), and enough **kubelet** instances on nodes to run workloads. The scheduler and controller-manager are needed for new scheduling/reconciliation, but existing pods can often keep running briefly without them. Networking (CNI plugin) must also be functional for pod-to-pod and Service traffic to work.

Q: What are Kubernetes node pools and what are they used for?

A: A node pool (or node group) is a set of nodes within a cluster that share the same configuration — instance type, labels, taints, autoscaling settings. They're used to segment workloads by hardware needs (e.g., a GPU node pool for ML workloads, a spot/preemptible pool for cost-sensitive batch jobs, a dedicated pool for system components), allowing different scaling policies and scheduling constraints per group instead of treating the whole cluster as homogeneous.

Q: Have you ever worked on a Kubernetes cluster upgrade?

A: This is an experience-based question — answer based on what you've actually done. If you have, describe the version jump, the tooling (eksctl, kubeadm, managed console), and how you validated it afterward.

Q: What do you check in the Kubernetes release notes before an upgrade?

A: Deprecated and removed API versions (so existing manifests don't break), breaking changes to core behavior, changes to default feature gates, changes affecting your CNI/CSI/Ingress controller compatibility, and any required minimum versions for add-ons. You'd also check `kubectl convert` or a deprecation-scanning tool (like `pluto` or `kubent`) against your manifests.

Q: What safety steps do you follow before upgrading a running production Kubernetes cluster?

A: Back up etcd, review release notes for breaking/deprecated API changes, test the upgrade in a staging/non-prod cluster first, ensure workloads have proper PodDisruptionBudgets and multiple replicas so draining doesn't cause downtime, confirm add-ons (CNI, CSI, Ingress controller, monitoring) are compatible with the target version, and schedule the upgrade during a low-traffic maintenance window with a rollback plan ready.

Q: Do you upgrade nodes one by one or all at once?

A: One at a time (rolling upgrade) is the standard, safe approach — you cordon and drain a single node, upgrade or replace it, verify it rejoins healthy, and only then move to the next. Upgrading all nodes simultaneously risks a full outage if something goes wrong, since there'd be no healthy capacity left to run workloads.

Q: If upgrading node by node, how do you evict/drain pods from a node?

A: First cordon the node so the scheduler stops placing new pods on it, then drain it, which safely evicts existing pods (respecting PodDisruptionBudgets) so they get rescheduled onto other healthy nodes before you take the node offline for upgrade.

Q: What command do you use to drain a Kubernetes node?

```
kubectl cordon <node-name>
kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data
```

A: After the upgrade, bring it back into rotation with `kubectl uncordon <node-name>` .

Q: If your cluster is on Kubernetes 1.31 and you need to upgrade to 1.33, how would you perform the upgrade?

A: Kubernetes only supports upgrading one minor version at a time, so you cannot jump straight from 1.31 to 1.33 — you must go 1.31 → 1.32 → 1.33 sequentially. For each hop: upgrade the control plane first (API server, then scheduler/controller-manager, then etcd if needed), verify cluster health, then upgrade nodes one at a time (cordon, drain, upgrade kubelet/kube-proxy or replace the node, uncordon), and finally upgrade add-ons/CNI if required for that version, before repeating the same process for the next minor version.

Q: You said you're using Blue-Green deployment. In production, how exactly does the traffic move from the old application to the new application?

A: In Blue-Green, two full environments exist simultaneously: "Blue" (current live version) and "Green" (new version), each fully deployed and tested independently. Once Green passes validation, traffic is cut over all at once by updating the routing layer — repointing the load balancer, Ingress, or Service selector from Blue to Green — rather than gradually shifting a percentage. If something goes wrong, you roll back instantly by pointing traffic back to Blue, since it's still fully running.

Q: Are you deploying two versions/applications and gradually dividing traffic between them?

A: That describes Canary, not Blue-Green — Blue-Green switches traffic all at once between two complete environments, while Canary gradually shifts a percentage of traffic (e.g., 5% → 25% → 50% → 100%) from the old version to the new one, monitoring metrics at each step before proceeding.

Q: How do you avoid downtime when deploying a new version?

A: Use a zero-downtime strategy such as Rolling Update (default in Kubernetes Deployments — replaces pods incrementally while keeping enough replicas available), Blue-Green (instant cutover between two fully live environments), or Canary (gradual traffic shift with monitoring). Combine this with readiness probes (so traffic isn't sent to a pod before it's ready), PodDisruptionBudgets (so voluntary disruptions like node drains don't take down too many replicas at once), and graceful shutdown handling (so in-flight requests finish before a pod terminates).

Terraform

Q: How good are you with Terraform?

A: This is an experience/self-assessment question — answer honestly based on your actual project work with modules, state management, and providers you've used.

Q: What Terraform work/scenarios have you handled in your project?

A: Answer based on your real experience — common examples include provisioning VPCs/networking, EC2/EKS clusters, S3 buckets, IAM roles/policies, RDS databases, managing remote state with locking, writing reusable modules, and integrating Terraform into CI/CD pipelines (Jenkins) with Vault-managed credentials.

Q: What happens when you run terraform init and terraform plan?

A: `terraform init` initializes the working directory: it downloads the required provider plugins, sets up the backend (e.g., S3 for remote state), and initializes any modules referenced in the configuration. `terraform plan` compares your configuration against the current state file and the real infrastructure, then produces an execution plan showing exactly what will be created, changed, or destroyed — without actually making any changes.

Q: How do you manage Terraform state files?

A: Use a remote backend (commonly S3 with DynamoDB for locking, or Terraform Cloud) instead of local state, so state is shared safely across a team, versioned, and protected from concurrent writes via state locking. Enable encryption at rest, restrict access via IAM, and never commit state files to version control since they can contain sensitive data.

Q: What are the different types of variables in Terraform?

A: Input variables (defined with `variable` blocks, parameterize the configuration), output variables (defined with `output` blocks, expose values after apply for use elsewhere), local values (`locals` , for computed/reused expressions within a module), and environment variables (`TF_VAR_<name>` , set outside the configuration to feed input variables).

Q: Can you write a Terraform configuration for an S3 bucket?

```
resource "aws_s3_bucket" "my_bucket" {
  bucket = "my-example-bucket-12345"

  tags = {
    Environment = "production"
  }
}

resource "aws_s3_bucket_versioning" "my_bucket_versioning" {
  bucket = aws_s3_bucket.my_bucket.id
  versioning_configuration {
    status = "Enabled"
  }
}
```

Q: Can you create multiple S3 buckets using for_each?

```
variable "bucket_names" {
  default = ["bucket-one", "bucket-two", "bucket-three"]
}

resource "aws_s3_bucket" "buckets" {
  for_each = toset(var.bucket_names)
  bucket   = each.value

  tags = {
    Name = each.value
  }
}
```

Q: What is Terraform state locking?

A: State locking prevents two people (or two pipeline runs) from running `terraform apply` against the same state simultaneously, which could corrupt the state file or cause conflicting infrastructure changes. When using an S3 backend, locking is typically implemented via a DynamoDB table — Terraform writes a lock entry before an operation and releases it afterward; if another run tries to start while a lock exists, it's blocked.

Q: How do you handle a Terraform state-locking situation?

A: First confirm whether the lock is legitimate — is another apply/plan genuinely in progress? If so, wait for it to finish. If the lock is stale (left behind by a crashed process or interrupted pipeline run), identify the lock holder/ID from the error message and, after confirming no process is actually running, use `terraform force-unlock` to release it.

Q: What are Terraform random providers/resources?

A: The `random` provider generates random values for use in configurations — e.g., `random_id`, `random_string`, `random_pet`, `random_password` — commonly used to create unique resource names (like appending a random suffix to a globally-unique S3 bucket name) or generate temporary passwords, without needing an external source of randomness.

Q: Suppose a Terraform pipeline crashes while holding the state lock. What happens?

A: The lock entry (e.g., in the DynamoDB lock table) remains in place even though the process that created it is gone, since nothing released it cleanly. This means the state stays locked indefinitely until someone manually intervenes — any subsequent `terraform plan` or `apply` will fail with a lock error until the stale lock is force-unlocked.

Q: If another DevOps engineer runs terraform apply while the state is locked, what error will they get?

A: Terraform returns a **"Error acquiring the state lock"** message, which includes details about the existing lock — typically the Lock ID, who/what holds it, the operation type, and the timestamp it was created — and refuses to proceed until the lock is released.

Q: How do you resolve the Terraform locked-state error?

A: Verify no other process is genuinely running against that state (check with your team, check the CI/CD pipeline status). If confirmed stale, note the Lock ID from the error output and run `terraform force-unlock <lock-id>` to manually clear it.

Q: What command do you use to force-unlock Terraform state?

```
terraform force-unlock <LOCK_ID>
```

Q: What do you check before running terraform force-unlock?

A: Confirm that no other terraform process (a teammate's session, or a CI/CD job) is actually running against that state right now — force-unlocking while a real operation is in progress can lead to two concurrent writes and state corruption. Check with the team, check pipeline job status/logs, and check how long the lock has been held (a lock held for hours with no active job is a strong signal it's stale).

Q: How do you identify who is holding/accessing the Terraform lock?

A: The lock error message itself includes metadata: the Lock ID, the operation being performed, who created it (username/hostname), and the creation timestamp. If using a DynamoDB lock table, you can also directly query the table for the lock item to see the same details.

Q: Can you write a Jenkins pipeline that runs all the Terraform commands?

```
pipeline {
  agent any
  environment {
    AWS_CREDS = credentials('aws-creds-id')
  }
  stages {
    stage('Init') {
      steps { sh 'terraform init' }
    }
    stage('Validate') {
      steps { sh 'terraform validate' }
    }
    stage('Plan') {
      steps { sh 'terraform plan -out=tfplan' }
    }
    stage('Approval') {
      steps { input message: 'Approve apply?' }
    }
    stage('Apply') {
      steps { sh 'terraform apply -auto-approve tfplan' }
    }
  }
}
```

Q: How is Terraform implemented in your production/development environment?

A: This is a project-experience question — describe your actual setup: whether Terraform runs via a Jenkins/CI pipeline with manual approval gates for prod, how modules are structured per environment (dev/stage/prod using separate state files or workspaces), and how state and credentials are managed.

Q: Do you run Terraform plan and apply manually or through Jenkins?

A: Best practice, and the typical production answer, is running both through a Jenkins (or other CI/CD) pipeline rather than manually from a laptop — this ensures consistency, an audit trail, and a mandatory approval gate before `apply`, especially for production. Manual runs are usually reserved for local development/testing against a dev environment.

Q: How are Terraform credentials handled in Jenkins?

A: Cloud credentials (AWS access keys, service principal secrets, etc.) are stored in Jenkins' built-in Credentials store (or better, pulled dynamically from a secrets manager like HashiCorp Vault) rather than hardcoded in the Jenkinsfile, and injected into the pipeline as environment variables at runtime using the `credentials()` binding or a Vault plugin step.

Q: How is HashiCorp Vault integrated with Jenkins for Terraform credentials?

A: The Jenkins Vault plugin (or a direct API call step) authenticates to Vault — commonly using AppRole or Kubernetes auth — and fetches short-lived, dynamically generated cloud credentials (e.g., AWS STS tokens via Vault's AWS secrets engine) at pipeline runtime. These are injected as environment variables for the duration of the Terraform stage and expire shortly after, avoiding long-lived static credentials sitting in Jenkins.

Q: How do you import an existing resource into Terraform?

```
terraform import aws_instance.my_ec2 i-0abcd1234efgh5678
```

A: This binds the existing AWS resource to a resource block already defined in your `.tf` configuration (`aws_instance.my_ec2` must exist in code first) — Terraform then records it in state, but you must still write the matching configuration attributes yourself so future plans don't show a diff.

Q: Suppose someone created an EC2 instance through the AWS UI. What steps do you follow before importing it into Terraform?

A: 1) Identify the resource's exact attributes (instance type, AMI, tags, security groups, subnet, etc.) from the console or via `aws ec2 describe-instances`. 2) Write a matching Terraform resource block in your configuration that mirrors those attributes as closely as possible. 3) Run `terraform import` to bring it under state management. 4) Run `terraform plan` immediately afterward to check for drift between your written config and the actual resource, and adjust the configuration until the plan shows no changes.

Q: What are the prerequisites before performing terraform import?

A: A resource block for that resource type must already exist in your `.tf` files (import only creates the state entry, not the configuration), you need the resource's unique identifier (e.g., instance ID, ARN), correct provider credentials/permissions to read the resource, and ideally a backup of the current state file in case something goes wrong.

Q: What does terraform import actually change?

A: It only updates the Terraform state file, adding a mapping between your resource address (e.g., `aws_instance.my_ec2`) and the real-world resource ID. It does not generate or modify any `.tf` configuration files — you're responsible for writing config that matches, and it does not alter the actual cloud resource in any way.

Q: Does Terraform import modify the existing cloud resource?

A: No. `terraform import` is a read-only operation against the cloud provider — it only reads the resource's current configuration to populate Terraform's state. The actual infrastructure resource itself is left completely untouched during the import; any changes would only happen later if you run `terraform apply` with a configuration that differs from the resource's real settings.